

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using Telerik.Web.UI;
using XSoft.Security;
using XSoft.Utility;
using WebForm.BusinessProcess;
using System.Web.UI.WebControls;
using System.Data;
using Common.Data.Hibernate;
namespace WebForm.Controls.CourseScheduling
{
    public static class RadComboBoxExtensions
    {
        public static void InitClassLessonPlace(this DropDownList cbx, string
termid)
        {
            string sql = "select id [Key],placename as [Value] from
ClassLessonPlace where schoolid='" + SecurityHolder.CurrentUser.SchoolId + "' and
termid='" + termid + "'";
            cbx.InitBySql(sql);
        }
        public static void InitModuleBySubject(this Ext.Net.Store store, string
subjectid)
        {
            string sql = string.Format(@"select id as [key],[abbreviation] [value]
from creditmodule where schoolid='{0}' and subjectid='{1}' and [name]<>''",
SecurityHolder.CurrentUser.SchoolId, subjectid);
            store.InitDataSet(sql);
        }
        public static void InitAllModule(this Ext.Net.Store store)
        {
            string sql = string.Format(@"select id as [key],[abbreviation] [value]
from creditmodule where schoolid='{0}' and type='0' and [name]<>''",
SecurityHolder.CurrentUser.SchoolId);
            store.InitDataSet(sql);
        }
        public static void InitModuleBySubject(this Ext.Net.ComboBox cbx, string
subjectid)
        {
            string sql = string.Format(@"select id as [key],[abbreviation] [value]
from creditmodule where schoolid='{0}' and subjectid='{1}' and type='0' and
[name]<>''", SecurityHolder.CurrentUser.SchoolId, subjectid);
            cbx.InitDataSet(sql);
        }
    }
}
#endregion
#region Equals And GetHashCode Overrides
public override bool Equals( object obj )
{

```

```

if( this == obj ) return true;
CategoryStock castObj = null;
try
{
castObj = (CategoryStock)obj;
} catch(Exception) { return false; }
return ( castObj != null ) &&
( this._id == castObj.Id );
}
public override int GetHashCode()
{

int hash = 57;
hash = 27 * hash * _id.GetHashCode();
return hash;
}
#endregion
#region ICloneable methods
public virtual object Clone()
{
return this.MemberwiseClone();
}
#endregion
}
#region Custom ICollection interface for CategoryStock
public interface ICategoryStockCollection : ICollection
{
CategoryStock this[int index]{get; set; }
void Add(CategoryStock pCategoryStock);
void Clear();
}
}
public CategoryStock this[int index]
{
get
{
return _arrayInternal[index];
}
set
{
_arrayInternal[index] = value;
}
}
public int Count { get { return _arrayInternal.Count; } }
public bool IsSynchronized { get { return false; } }
public object SyncRoot { get { return _arrayInternal; } }
public void CopyTo(Array array, int
index){ _arrayInternal.CopyTo((CategoryStock[])array, index); }
public IEnumerator GetEnumerator() { return _arrayInternal.GetEnumerator(); }
public void Add(CategoryStock pCategoryStock)

```

```
{ _arrayInternal.Add(pCategoryStock); }
public void Clear() { _arrayInternal.Clear(); }
public IList<CategoryStock> GetList() { return _arrayInternal; }
}
#endregion
}
[Serializable]
public class CategoryUse : ICloneable, ICategoryUse
{
    #region Member Variables
    protected string _id;
    protected int? _num;
    protected DateTime? _dodate;
    protected string _description;
    protected string _remark;
    protected string _state;
    protected string _categoryid;
    protected string _useplace;
    protected string _stockid;
    protected string _stockname;
    protected string _usedeptid;
    protected string _usedeptname;
    protected string _userid;
    protected string _username;
    protected string _useplaceid;
    protected bool _bIsDeleted;
    protected bool _bIsChanged;
    #endregion
    #region Constructors
    public CategoryUse() {}
    public CategoryUse(string pId, int? pNum, DateTime? pDodate, string pDescription,
        string pRemark, string pState, string pCategoryid, string pUseplace, string
        pStockid, string pStockname, string pUsedeptid, string pUsedeptname, string pUserid,
        string pUsername, string pUseplaceid)
    {
        this._id = pId;
        this._num = pNum;
        this._dodate = pDodate;
        this._description = pDescription;
        this._remark = pRemark;
        this._state = pState;
        this._categoryid = pCategoryid;
        this._useplace = pUseplace;
        this._stockid = pStockid;
        this._stockname = pStockname;
        this._usedeptid = pUsedeptid;
        this._usedeptname = pUsedeptname;
        this._userid = pUserid;
        this._username = pUsername;
        this._useplaceid = pUseplaceid;
    }
}
```

```
}
public CategoryUse(string pId)
{
    this._id = pId;
}
#endregion
#region Public Properties
public virtual string Id
{
    get { return _id; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Id", "Id value, cannot contain more than
50 characters");
        _bIsChanged |= (_id != value);
        _id = value;
    }
}
public virtual int? Num
{
    get { return _num; }
    set { _bIsChanged |= (_num != value); _num = value; }
}
public virtual DateTime? Dodate
{
    get { return _dodate; }
    set { _bIsChanged |= (_dodate != value); _dodate = value; }
}
public virtual string Description
{
    get { return _description; }
    set
    {
        if (value != null && value.Length > 500)
            throw new ArgumentOutOfRangeException("Description", "Description value,
cannot contain more than 500 characters");
        _bIsChanged |= (_description != value);
        _description = value;
    }
}
public virtual string Remark
{
    get { return _remark; }
    set
    {
        if (value != null && value.Length > 500)
            throw new ArgumentOutOfRangeException("Remark", "Remark value, cannot contain
more than 500 characters");
        _bIsChanged |= (_remark != value);
    }
}
```

```
    _remark = value;
}
}
public virtual string State
{
    get { return _state; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("State", "State value, cannot contain more than 50 characters");
        _bIsChanged |= (_state != value);
        _state = value;
    }
}
public virtual string Categoryid
{
    get { return _categoryid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Categoryid", "Categoryid value, cannot contain more than 50 characters");
        _bIsChanged |= (_categoryid != value);
        _categoryid = value;
    }
}
public virtual string Useplace
{
    get { return _useplace; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Useplace", "Useplace value, cannot contain more than 50 characters");
        _bIsChanged |= (_useplace != value);
        _useplace = value;
    }
}
public virtual string Stockid
{
    get { return _stockid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Stockid", "Stockid value, cannot contain more than 50 characters");
        _bIsChanged |= (_stockid != value);
        _stockid = value;
    }
}
```

```
}
public virtual string Stockname
{
    get { return _stockname; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Stockname", "Stockname value, cannot contain more than 50 characters");
        _bIsChanged |= (_stockname != value);
        _stockname = value;
    }
}
public virtual string Usedeptid
{
    get { return _usedeptid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Usedeptid", "Usedeptid value, cannot contain more than 50 characters");
        _bIsChanged |= (_usedeptid != value);
        _usedeptid = value;
    }
}
public virtual string Usedeptname
{
    get { return _usedeptname; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Usedeptname", "Usedeptname value, cannot contain more than 50 characters");
        _bIsChanged |= (_usedeptname != value);
        _usedeptname = value;
    }
}
public virtual string Userid
{
    get { return _userid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Userid", "Userid value, cannot contain more than 50 characters");
        _bIsChanged |= (_userid != value);
        _userid = value;
    }
}
public virtual string Username
```

```
{
get { return _username; }
set
{
    if (value != null && value.Length > 50)
        throw new ArgumentOutOfRangeException("Username", "Username value, cannot
contain more than 50 characters");
    _bIsChanged |= (_username != value);
    _username = value;
}
}
public virtual string Useplaceid
{
get { return _useplaceid; }
set
{
    if (value != null && value.Length > 50)
        throw new ArgumentOutOfRangeException("Useplaceid", "Useplaceid value, cannot
contain more than 50 characters");
    _bIsChanged |= (_useplaceid != value);
    _useplaceid = value;
}
}
public virtual bool IsDeleted
{
get
{
return _bIsDeleted;
}
set
{
    _bIsDeleted = value;
}
}
public virtual bool IsChanged
{
get
{
return _bIsChanged;
}
set
{
    _bIsChanged = value;
}
}
}
#endregion
#region Equals And GetHashCode Overrides
public override bool Equals( object obj )
{
    if( this == obj ) return true;
```

```
CategoryUse castObj = null;
try
{
    castObj = (CategoryUse)obj;
} catch(Exception) { return false; }
return ( castObj != null ) &&
( this._id == castObj.Id );
}

public override int GetHashCode()
{

    int hash = 57;
    hash = 27 * hash * _id.GetHashCode();
    return hash;
}
#endregion
#region ICloneable methods
public virtual object Clone()
{
    return this.MemberwiseClone();
}
#endregion
}

#region Custom ICollection interface for CategoryUse
public interface ICategoryUseCollection : ICollection
{
    CategoryUse this[int index]{get; set; }
    void Add(CategoryUse pCategoryUse);
    void Clear();
}
[Serializable]
public class CategoryUseCollection : ICategoryUseCollection
{
    private IList<CategoryUse> _arrayInternal;
    public CategoryUseCollection()
    {
        _arrayInternal = new List<CategoryUse>();
    }
    public CategoryUseCollection( IList<CategoryUse> pSource )
    {
        _arrayInternal = pSource;
        if(_arrayInternal == null)
        {
            _arrayInternal = new List<CategoryUse>();
        }
    }
    public CategoryUse this[int index]
    {
        get
        {
```



```

return _arrayInternal[index];
}
set
{
_arrayInternal[index] = value;
}
}

public int Count { get { return _arrayInternal.Count; } }
public bool IsSynchronized { get { return false; } }
public object SyncRoot { get { return _arrayInternal; } }
public void CopyTo(Array array, int index){ _arrayInternal.CopyTo((CategoryUse[])array, index); }
public IEnumerator GetEnumerator() { return _arrayInternal.GetEnumerator(); }
public void Add(CategoryUse pCategoryUse) { _arrayInternal.Add(pCategoryUse); }
public void Clear() { _arrayInternal.Clear(); }
public IList<CategoryUse> GetList() { return _arrayInternal; }
}
#endregion
}

public static void InitModuleMultiCombo(this Ext.Net.MultiCombo cbx)
{
    cbx.Items.Clear();
    string sql = string.Format(@"select id as [key],[abbreviation] [value]
from creditmodule where schoolid='{0}' and type='0' and [name]<>'I' order by
id,abbreviation", SecurityHolder.CurrentUser.SchoolId);
    DataTable dt = SessionHolder.CreateTemplate().ExecuteDataTable(sql);
    foreach (DataRow row in dt.Rows)
    {
        cbx.Items.Add(new Ext.Net.ListItem(row["value"].ToString(),
row["id"].ToString()));
    }
}

public static void InitCurrentClassPlace(this DropDownList ddl)
{
    string sql = "select id [Key],placename as [Value] from
ClassLessonPlace where schoolid='" + SecurityHolder.CurrentUser.SchoolId + "' and
termid='" + commonFunction.getCurrentTermID() + "' and iscurrent='1'";
    ddl.InitBySql(sql);
}

public static void BindClassPlace(this DropDownList ddl)
{
    string sql = "select id [Key],placename as [Value] from
ClassLessonPlace where schoolid='" + SecurityHolder.CurrentUser.SchoolId + "' and
termid='" + commonFunction.getCurrentTermID() + "'";
    ddl.InitBySql(sql);
}

public static void InitClassLessonTemp(this RadComboBox cbx, string
placeId)
{
    if (placeId.IsNullOrEmpty()) return;

```

```

        string sql = "select a.classid [Key], a.classname [Value] from classinfo
a, classlesson temp b where a.classid=b.classid and b.placeid=" + placeId + " group
by a.classid , a.classname";
        cbx.InitBySql(sql);
    }
    public static void BindTakeLessonKind(this DropDownList ddl)
    {
        string sql = "select id [Key], Name as [Value] from TakeLessonKind where
schoolid=' " + SecurityHolder.CurrentUser.SchoolId + "' ";
        ddl.InitBySql(sql);
    }
}

public static void InitFieldSubjectModule(this RadTreeView rvtModule, string
termId)
{
    rvtModule.Nodes.Clear();
    List<OrmLib.CreditField> cfs =
SessionHolder.Current.Linq<OrmLib.CreditField>().Where(c => c.SchoolId ==
SecurityHolder.CurrentUser.SchoolId).ToList();
    RadTreeNode rootnode = new
RadTreeNode(SecurityHolder.CurrentUser.SchoolInfo.SchoolName, termId);
    rootnode.ImageUrl = "~/images/common/house.png";
    rvtModule.Nodes.Add(rootnode);
    foreach (OrmLib.CreditField cf in cfs)
    {
        RadTreeNode tn = new RadTreeNode();
        tn.Text = cf.Name + "";
        tn.Value = cf.Id.ToString();
        tn.ImageUrl = "~/images/common/folder-open.gif";
        InitSubject(tn, cf.Id.ToString(), termId);
        rootnode.Nodes.Add(tn);
    }
    if (rvtModule.Nodes.Count <= 0) return;
    rvtModule.Nodes[0].Expanded = true;
}

namespace WebForm.Controls.DeptManger
{
    public static class DeptExtend
    {
        public static void BindDept(this RadTreeView tv)
        {
            tv.BindDept("-1");
        }

        public static void BindDept(this RadTreeView tv, string pdeptid)
        {
            tv.Nodes.Clear();
            string sql = string.Format(@"select depid, deptname from department
where depid={0}", pdeptid);

```

```

        DataTable dt = SessionHolder.CreateTemplate().ExecuteDataTable(sql);
        if (dt == null || dt.Rows.Count == 0)
        {
            return;
        }
        RadTreeNode tn = new RadTreeNode();
        tn.Text = dt.Rows[0]["deptname"].ToString();
        tn.Value = dt.Rows[0]["depid"].ToString();
        tv.Nodes.Add(tn);
        sql = string.Format(@"with myT as(select depid,deptname,parentid from
department ),
                                myT2 as(select * from myT where parentid=' {0}'
                                union all
                                select myT.* from myT2 inner join myT
                                on myT2.depid=myT.parentid
                                )
                                select depid,deptname,parentid from myT2",
pdeptid);
        dt = SessionHolder.CreateTemplate().ExecuteDataTable(sql);
        BindSunDepartment(tn, dt);
        tv.ExpandAllNodes();
    }
    private static void BindSunDepartment(RadTreeNode tn, DataTable dt)
    {
        foreach (DataRow dr in dt.Select(string.Format("parentid=' {0}' ",
tn.Value)))
        {
            RadTreeNode stn = new RadTreeNode();
            stn.Text = dr["deptname"].ToString();
            stn.Value = dr["depid"].ToString();
            tn.Nodes.Add(stn);
            BindSunDepartment(stn, dt);
        }
    }
}
[Serializable]
public class ClassAllCocktail : ICloneable, IClassAllCocktail
{
    #region Member Variables
    protected int _id;
    protected string _usercode;
    protected string _username;
    protected string _usertype;
    protected string _maxim;
    protected DateTime? _date;
    protected string _classid;
    protected string _schoolid;
    protected bool _bIsDeleted;
    protected bool _bIsChanged;
    #endregion
    #region Constructors

```

```
public ClassAllCocktail() {}
public ClassAllCocktail(string pUserCode, string pUserName, string pUserType,
string pMaxim, DateTime? pDate, string pClassId, string pSchoolId)
{
    this._usercode = pUserCode;
    this._username = pUserName;
    this._usertype = pUserType;
    this._maxim = pMaxim;
    this._date = pDate;
    this._classid = pClassId;
    this._schoolid = pSchoolId;
}
public ClassAllCocktail(int pId)
{
    this._id = pId;
}
#endregion
#region Public Properties
public virtual int Id
{
    get { return _id; }
    set { _bIsChanged |= (_id != value); _id = value; }
}
public virtual string UserCode
{
    get { return _usercode; }
    set
    {
        if (value != null && value.Length > 100)
            throw new ArgumentOutOfRangeException("UserCode", "UserCode value, cannot
contain more than 100 characters");
        _bIsChanged |= ();
        _usercode = value;
    }
}
public virtual string UserName
{
    get { return _username; }
    set
    {
        if (value != null && value.Length > 100)
            throw new ArgumentOutOfRangeException("UserName", "UserName value, cannot
contain more than 100 characters");
        _bIsChanged |= (_username != value);
        _username = value;
    }
}
public virtual string UserType
{
    get { return _usertype; }
```

```
set
{
    if (value != null && value.Length > 100)
        throw new ArgumentOutOfRangeException("UserType", "UserType value, cannot contain more than 100 characters");
    _bIsChanged |= (_usertype != value);
    _usertype = value;
}
}

public virtual string Maxim
{
    get { return _maxim; }
    set
    {
        if (value != null && value.Length > 2147483647)
            throw new ArgumentOutOfRangeException("Maxim", "Maxim value, cannot contain more than 2147483647 characters");
        _bIsChanged |= (_maxim != value);
        _maxim = value;
    }
}

public virtual DateTime? Date
{
    get { return _date; }
    set { _bIsChanged |= (_date != value); _date = value; }
}

public virtual string ClassId
{
    get { return _classid; }
    set
    {
        if (value != null && value.Length > 100)
            throw new ArgumentOutOfRangeException("ClassId", "ClassId value, cannot contain more than 100 characters");
        _bIsChanged |= ();
        _classid = value;
    }
}

public class ClassBigEvent : ICloneable, IClassBigEvent
{
    #region Member Variables
    protected int _id;
    protected string _classid;
    protected string _termid;
    protected string _content;
    protected string _schoolid;
    protected string _title;
    protected DateTime? _date;
    protected string _usercode;
    protected string _username;
```

```
protected bool _bIsDeleted;
protected bool _bIsChanged;
#endregion
#region Constructors
public ClassBigEvent() {}
public ClassBigEvent(string pClassId, string pTermId, string pConenent, string
pSchoolId, string pTitle, DateTime? pDate, string pUserCode, string pUserName)
{
    this._classid = pClassId;
    this._termid = pTermId;
    this._conenent = pConenent;
    this._schoolid = pSchoolId;
    this._title = pTitle;
    this._date = pDate;
    this._usercode = pUserCode;
    this._username = pUserName;
}
public ClassBigEvent(int pId)
{
    this._id = pId;
}
#endregion
#region Public Properties
public virtual int Id
{
    get { return _id; }
    set { _bIsChanged |= (_id != value); _id = value; }
}
public virtual string ClassId
{
    get { return _classid; }
    set
    {
        if (value != null && value.Length > 32)
            throw new ArgumentOutOfRangeException("ClassId", "ClassId value, cannot
contain more than 32 characters");
        _bIsChanged |= (_classid != value);
        _classid = value;
    }
}
public virtual string TermId
{
    get { return _termid; }
    set
    {
        if (value != null && value.Length > 32)
            throw new ArgumentOutOfRangeException("TermId", "TermId value, cannot contain
more than 32 characters");
        _bIsChanged |= (_termid != value);
        _termid = value;
    }
}
```

```
}
}
public virtual string Contenet
{
    get { return _content; }
    set
    {
        if (value != null && value.Length > 2000)
            throw new ArgumentOutOfRangeException("Content", "Content value, cannot contain more than 2000 characters");
        _bIsChanged |= (_content != value);
        _content = value;
    }
}
public virtual string SchoolId
{
    get { return _schoolid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("SchoolId", "SchoolId value, cannot contain more than 50 characters");
        _bIsChanged |= (_schoolid != value);
        _schoolid = value;
    }
}
public virtual string Title
{
    get { return _title; }
    set
    {
        if (value != null && value.Length > 500)
            throw new ArgumentOutOfRangeException("Title", "Title value, cannot contain more than 500 characters");
        _bIsChanged |= (_title != value);
        _title = value;
    }
}
public virtual DateTime? Date
{
    get { return _date; }
    set { _bIsChanged |= (_date != value); _date = value; }
}
public virtual string UserCode
{
    get { return _usercode; }
    set
    {
        if (value != null && value.Length > 500)
            throw new ArgumentOutOfRangeException("UserCode", "UserCode value, cannot
```

```
contain more than 500 characters");
    _bIsChanged |= (_usercode != value);
    _usercode = value;
}
}
public virtual string UserName
{
    get { return _username; }
    set
    {
        if (value != null && value.Length > 500)
            throw new ArgumentOutOfRangeException("UserName", "UserName value, cannot
contain more than 500 characters");
        _bIsChanged |= (_username != value);
        _username = value;
    }
}
public class ClassJournal : ICloneable, IClassJournal
{
    #region Member Variables
    protected int _cjid;
    protected int? _ctid;
    protected int? _studentcount;
    protected int? _girlcount;
    protected int? _minoritycount;
    protected string _monitorname;
    protected string _classhonorary;
    protected bool _bIsDeleted;
    protected bool _bIsChanged;
    #endregion
    #region Constructors
    public ClassJournal() {}
    public ClassJournal(int? pCtId, int? pStudentCount, int? pGirlCount, int?
pMinorityCount, string pMonitorName, string pClassHonorary)
    {
        this._ctid = pCtId;
        this._studentcount = pStudentCount;
        this._girlcount = pGirlCount;
        this._minoritycount = pMinorityCount;
        this._monitorname = pMonitorName;
        this._classhonorary = pClassHonorary;
    }
    #endregion
    #region Public Properties
    public virtual int CjId
    {
        get { return _cjid; }
        set { _bIsChanged |= (_cjid != value); _cjid = value; }
    }
    public virtual int? CtId
```



```
{
get { return _ctid; }
set { _bIsChanged |= (_ctid != value); _ctid = value; }
}

public virtual int? StudentCount
{
get { return _studentcount; }
set { _bIsChanged |= (_studentcount != value); _studentcount = value; }
}

public virtual int? GirlCount
{
get { return _girlcount; }
set { _bIsChanged |= (_girlcount != value); _girlcount = value; }
}

public virtual int? MinorityCount
{
get { return _minoritycount; }
set { _bIsChanged |= (_minoritycount != value); _minoritycount = value; }
}

public virtual string MonitorName
{
get { return _monitorname; }
set
{
if (value != null && value.Length > 10)
throw new ArgumentOutOfRangeException("MonitorName", "MonitorName value,
cannot contain more than 10 characters");
_bIsChanged |= (_monitorname != value);
_monitorname = value;
}
}

public virtual string ClassHonorary
{
get { return _classhonorary; }
set
{
if (value != null && value.Length > 100)
throw new ArgumentOutOfRangeException("ClassHonorary", "ClassHonorary value,
cannot contain more than 100 characters");
_bIsChanged |= (_classhonorary != value);
_classhonorary = value;
}
}

#endregion
#region Equals And GetHashCode Overrides
public override bool Equals( object obj )
{
if( this == obj ) return true;
ClassJournal castObj = null;
try
```

```
{
castObj = (ClassJournal)obj;
} catch(Exception) { return false; }
return ( castObj != null ) &&
( this._cjid == castObj.CjId );
}

public override int GetHashCode()
{

int hash = 57;
hash = 27 * hash * _cjid.GetHashCode();
return hash;
}

#endregion
#region ICloneable methods
public virtual object Clone()
{
return this.MemberwiseClone();
}
#endregion
}

#region Custom ICollection interface for ClassJournal
public interface ICollection : ICollection
{
ClassJournal this[int index]{get; set; }
void Add(ClassJournal pClassJournal);
void Clear();
}

[Serializable]
public class ClassJournalCollection : ICollection
{
private IList<ClassJournal> _arrayInternal;
public ClassJournalCollection()
{
_arrayInternal = new List<ClassJournal>();
}

public ClassJournalCollection( IList<ClassJournal> pSource )
{
_arrayInternal = pSource;
if(_arrayInternal == null)
{
_arrayInternal = new List<ClassJournal>();
}
}

public ClassJournal this[int index]
{
get
{
return _arrayInternal[index];
}
}
```

```

set
{
    _arrayInternal[index] = value;
}
}

public int Count { get { return _arrayInternal.Count; } }
public bool IsSynchronized { get { return false; } }
public object SyncRoot { get { return _arrayInternal; } }
public void CopyTo(Array array, int index) { _arrayInternal.CopyTo((ClassJournal[])array, index); }
public IEnumerator GetEnumerator() { return _arrayInternal.GetEnumerator(); }
public void Add(ClassJournal pClassJournal) { _arrayInternal.Add(pClassJournal); }
public void Clear() { _arrayInternal.Clear(); }
public IList<ClassJournal> GetList() { return _arrayInternal; }
}
#endregion
}
namespace WebForm.Controls.DeptManger
{
    public static class DeptExtend
    {
        public static void BindDept(this RadTreeView tv)
        {
            tv.BindDept("-1");
        }

        public static void BindDept(this RadTreeView tv, string pdeptid)
        {
            tv.Nodes.Clear();
            string sql = string.Format(@"select depid,deptname from department
where depid='{0}',", pdeptid);
            DataTable dt = SessionHolder.CreateTemplate().ExecuteDataTable(sql);
            if (dt == null || dt.Rows.Count == 0)
            {
                return;
            }
            RadTreeNode tn = new RadTreeNode();
            tn.Text = dt.Rows[0]["deptname"].ToString();
            tn.Value = dt.Rows[0]["depid"].ToString();
            tv.Nodes.Add(tn);
            sql = string.Format(@"with myT as(select depid,deptname,parentid from
department ),
                                myT2 as(select * from myT where parentid='{0}'
                                union all
                                select myT.* from myT2 inner join myT
                                on myT2.depid=myT.parentid
                                )
                                select depid,deptname,parentid from myT2",
                                pdeptid);
            dt = SessionHolder.CreateTemplate().ExecuteDataTable(sql);

```

```

        BindSunDepartment(tn, dt);
        tv.ExpandAllNodes();
    }
    private static void BindSunDepartment(RadTreeNode tn, DataTable dt)
    {
        foreach (DataRow dr in dt.Select(string.Format("parentid='{0}'",
tn.Value)))
        {
            RadTreeNode stn = new RadTreeNode();
            stn.Text = dr["deptname"].ToString();
            stn.Value = dr["depid"].ToString();
            tn.Nodes.Add(stn);
            BindSunDepartment(stn, dt);
        }
    }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Web.UI.WebControls;
using XSoft.Security;
namespace WebForm.Controls.DormitoryMgr
{
    public static class DormitoryExt
    {
        {
            public static void BindDormitoryKind(this DropDownList ddl)
            {
                ddl.Items.Clear();
                string sql = "select Id as [Key], [Name] as [Value] from DormitoryMgrKind
where SchoolId=' " + SecurityHolder.CurrentUser.SchoolId + "'";
                if (SecurityHolder.CurrentUser.UserType == UserType.Teacher)
                {
                    sql += " and charindex(' " + SecurityHolder.CurrentUser.Id +
"", usercode) > 0";
                }
                ddl.InitBySql(sql);
            }
            public static void BindDormitoryAllKind(this DropDownList ddl)
            {
                ddl.Items.Clear();
                string sql = "select Id as [Key], [Name] as [Value] from DormitoryMgrKind
where SchoolId=' " + SecurityHolder.CurrentUser.SchoolId + "'";
                ddl.InitBySql(sql);
            }
        }
    }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

```

```

using Telerik.Web.UI;
using XSoft.Security;
using System.Web.UI.WebControls;
using Common.Data.Hibernate;
using XSoft.CommonBiz;
using WebForm.Common;
using NHibernate.Linq;
namespace WebForm.Controls.EcManager
{
    public static class RadComboBoxExtensions
    {
        public static void BindEduTermInfo(this RadComboBox radCombox, string
educationId, bool isSelCurrentTerm)
        {
            string sql = string.Format(@"select termid as [key], termName as [value]
from TermInfo_edu where educationid='{0}'
order by schoolyear,term", educationId);
            radCombox.InitBySql(sql);
            if (isSelCurrentTerm)
            {
                radCombox.SelectedValue =
XSoft.CommonBiz.EduBaseManager.GetCurEduTermId(educationId);
            }
        }

        public static void BindEduTermInfo(this DropDownList ddl, string
educationId, bool isSelCurrentTerm)
        {
            string sql = string.Format(@"select termid as [key], termName as [value]
from TermInfo_edu where educationid='{0}'
order by schoolyear,term", educationId);
            ddl.InitBySql(sql);
            if (isSelCurrentTerm)
            {
                ddl.SelectedValue =
XSoft.CommonBiz.EduBaseManager.GetCurEduTermId(educationId);
            }
        }

        public static void BindBaseGradeInfo(this RadComboBox radCombox)
        {
            string sql = string.Format(@"select a.sectionname+a.gradename as
value,
                                cast(a.section
                                as
varchar)+cast(a.gradeindex as varchar) as text from GradeReportTemp a
                                group
                                by
a.sectionname,a.gradename,a.section,a.gradeindex
                                order by a.section,a.gradeindex");
            DataMorph.CreateInstance.InitRadComboBox(ref radCombox, sql);
        }

        public static void BindBaseGradeInfo(this DropDownList radCombox)
        {

```

```

        string sql = string.Format(@"select a.sectionname+a.gradename as text,
                                   cast(a.section                               as
varchar)+cast(a.gradeindex as varchar) as value from GradeReportTemp a
                                   group                                     by
a.sectionname, a.gradename, a.section, a.gradeindex
                                   order by a.section, a.gradeindex");
        DataMorph.CreateInstance.InitRadComboBox(ref radCombox, sql);
        public static void BindEduSchoolList(this CheckBoxList cbl, string
educationid, string section)
        {
            string schoolstr =
XSoft.CommonBiz.EduBaseManager.GetSchoolStrByEdu(educationid);
            string sql = string.Format(@"select b.schoolid as
[key], b.schoolname as [value]
from schoolinfo b where b.schoolid in ({0}) and b.schoolid
in
(select schoolid from schoolsection where section='{1}'
and isused='1' )", schoolstr, section);

            cbl.InitBySql(sql);
        }
        public static void BindEduSchool(this DropDownList cbl, string
educationid)
        {
            string schoolstr =
XSoft.CommonBiz.EduBaseManager.GetSchoolStrByEdu(educationid);
            string sql = string.Format(@"select b.schoolid as [key], b.schoolname
as [value]
from schoolinfo b where b.schoolid in ({0}) ", schoolstr);
            cbl.InitBySql(sql);
        }
        public static void BindEduSchoolList(this RadComboBox cbl, string
educationid)
        {
            string allschoolstr =
XSoft.CommonBiz.EduBaseManager.GetSchoolStrByEdu(educationid);
            string sql = string.Format(@"select b.schoolid as [key], b.schoolname
as [value]
from schoolinfo b where b.schoolid in ({0})",
allschoolstr);
            cbl.InitBySql(sql);
        }
        public static void BindEduExam(this RadComboBox cbl, string termid)
        {
            string sql = string.Format(@"select examid as [key], examname as [value]
from examname_edu where termid='{0}' ", termid);
            cbl.InitBySql(sql);
        }
        public static void BindEduExam(this DropDownList cbl, string termid)
        {

```

```

        string sql = string.Format(@"select examid as [key], examname as [value]
from examname_edu where termid='{0}', termid);
        cbl.InitBySql(sql);
    }

    public static void BindEduSubject(this DropDownList cbl, string eduid)
    {
        string sql = string.Format(@"select code as [key], name as [value] from
Dm_Subject_Education where EducationId='{0}' order by code", eduid);
        cbl.InitBySql(sql);
    }

    public static void BindEduSubject(this CheckedList cbl, string eduid)
    {
        string sql = string.Format(@"select code as [key], name as [value] from
Dm_Subject_Education where EducationId='{0}' order by code", eduid);
        cbl.InitBySql(sql);
    }

    public static void BindEduExamSchool(this RadComboBox cbl, string examid)
    {
        string sql = string.Format(@"select a.schoolid as [key], b.schoolname
as[value] from  dbo.ExamSchoolDetails_edu a, schoolinfo b where a.examid='{0}' and
a.schoolid=b.schoolid", examid);
        cbl.InitBySql(sql);
        cbl.Items.Insert(0, new RadComboBoxItem("", "-1"));
    }

    public static void BindEduExamSchool(this DropDownList cbl, string examid)
    {
        string sql = string.Format(@"select a.schoolid as [key], b.schoolname
as[value] from  dbo.ExamSchoolDetails_edu a, schoolinfo b where a.examid='{0}' and
a.schoolid=b.schoolid", examid);
        cbl.InitBySql(sql);
        cbl.Items.Insert(0, new ListItem("", "-1"));
    }

    public static void BindEduExamSubject(this RadComboBox cbl, string examid,
bool isall)
    {
        string sql = string.Format(@"select a.subjectcode [key], b.[name]
[value] from ExamSubjectDetails_edu a,
                                dm_subject_education b , examname_edu c
where a.examid=c.examid and a.examid='{0}'
                                and b.educationid=c.educationid and
a.subjectcode=b.code", examid);
        cbl.InitBySql(sql);
        if (isall)
        {
            cbl.Items.Insert(0, new RadComboBoxItem("", "*"));
        }
    }

    public static void BindEduExamSubject(this DropDownList cbl, string examid,
bool isall)

```

```

        {
            string sql = string.Format(@"select a.subjectcode [key], b.[name]
[value] from ExamSubjectDetails_edu a,
                                dm_subject_education b ,examname_edu c
where a.examid=c.examid and a.examid='{0}'
                                and b.educationid=c.educationid and
a.subjectcode=b.code", examid);
            cbl.InitBySql(sql);
            if (isall)
            {
                cbl.Items.Insert(0, new ListItem("", "*"));
            }
        }
    }
}

[Serializable]
public class ClassLesson : ICloneable, IClassLesson
{
    #region Member Variables
    protected int _clid;
    protected int? _dayofweek;
    protected int? _numberoflesson;
    protected string _subjectid;
    protected string _subjectname;
    protected string _moduleid;
    protected string _modulename;
    protected string _teacherid;
    protected string _teachername;
    protected DateTime? _startdate;
    protected string _venue;
    protected byte? _ptimes;
    protected string _fix;
    protected byte? _intime;
    protected string _studentassessment;
    protected string _termid;
    protected string _classid;
    protected string _code;
    protected bool _bIsDeleted;
    protected bool _bIsChanged;
    #endregion
    #region Constructors
    public ClassLesson() {}
    public ClassLesson(int? pDayOfWeek, int? pNumberOfLesson, string pSubjectId,
string pSubjectname, string pModuleid, string pModulename, string pTeacherId,
string pTeacherName, DateTime? pStartDate, string pVenue, byte? pPtimes, string
pFix, byte? pIntime, string pStudentAssessment, string pTermid, string pClassid,
string pCode)
    {
        this._dayofweek = pDayOfWeek;
        this._numberoflesson = pNumberOfLesson;
    }
}

```



```
this._subjectid = pSubjectId;
this._subjectname = pSubjectname;
this._moduleid = pModuleid;
this._modulename = pModulename;
this._teacherid = pTeacherId;
this._teachername = pTeacherName;
this._startdate = pStartdate;
this._venue = pVenue;
this._ptimes = pPtimes;
this._fix = pFix;
this._intime = pIntime;
this._studentassessment = pStudentAssessment;
this._termid = pTermid;
this._classid = pClassid;
this._code = pCode;
}
public ClassLesson(int pClId)
{
    this._clid = pClId;
}
#endregion
#region Public Properties
public virtual int ClId
{
    get { return _clid; }
    set { _bIsChanged |= (_clid != value); _clid = value; }
}
public virtual int? DayOfWeek
{
    get { return _dayofweek; }
    set { _bIsChanged |= (_dayofweek != value); _dayofweek = value; }
}
public virtual int? NumberOfLesson
{
    get { return _numberoflesson; }
    set { _bIsChanged |= (_numberoflesson != value); _numberoflesson = value; }
}
public virtual string SubjectId
{
    get { return _subjectid; }
    set
    {
        if (value != null && value.Length > 1000)
            throw new ArgumentOutOfRangeException("SubjectId", "SubjectId value, cannot
contain more than 1000 characters");
        _bIsChanged |= (_subjectid != value);
        _subjectid = value;
    }
}
public virtual string Subjectname
```

```
{
get { return _subjectname; }
set
{
    if (value != null && value.Length > 1000)
        throw new ArgumentOutOfRangeException("Subjectname", "Subjectname value,
cannot contain more than 1000 characters");
    _bIsChanged |= (_subjectname != value);
    _subjectname = value;
}
}
public virtual string Moduleid
{
get { return _moduleid; }
set
{
    if (value != null && value.Length > 50)
        throw new ArgumentOutOfRangeException("Moduleid", "Moduleid value, cannot
contain more than 50 characters");
    _bIsChanged |= (_moduleid != value);
    _moduleid = value;
}
}
public virtual string Modulename
{
get { return _modulename; }
set
{
    if (value != null && value.Length > 50)
        throw new ArgumentOutOfRangeException("Modulename", "Modulename value, cannot
contain more than 50 characters");
    _bIsChanged |= (_modulename != value);
    _modulename = value;
}
}
public virtual string TeacherId
{
get { return _teacherid; }
set
{
    if (value != null && value.Length > 1000)
        throw new ArgumentOutOfRangeException("TeacherId", "TeacherId value, cannot
contain more than 1000 characters");
    _bIsChanged |= (_teacherid != value);
    _teacherid = value;
}
}
public virtual string TeacherName
{
get { return _teachername; }
```

```
set
{
    if (value != null && value.Length > 1000)
        throw new ArgumentOutOfRangeException("TeacherName", "TeacherName value,
cannot contain more than 1000 characters");
    _bIsChanged |= (_teachername != value);
    _teachername = value;
}
}

public virtual DateTime? Startdate
{
    get { return _startdate; }
    set { _bIsChanged |= (_startdate != value); _startdate = value; }
}

public virtual string Venue
{
    get { return _venue; }
    set
    {
        if (value != null && value.Length > 255)
            throw new ArgumentOutOfRangeException("Venue", "Venue value, cannot contain
more than 255 characters");
        _bIsChanged |= (_venue != value);
        _venue = value;
    }
}

public virtual byte? Ptimes
{
    get { return _ptimes; }
    set { _bIsChanged |= (_ptimes != value); _ptimes = value; }
}

public virtual string Fix
{
    get { return _fix; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Fix", "Fix value, cannot contain more
than 50 characters");
        _bIsChanged |= (_fix != value);
        _fix = value;
    }
}

public virtual byte? Intime
{
    get { return _intime; }
    set { _bIsChanged |= (_intime != value); _intime = value; }
}

public virtual string StudentAssessment
{

```

```
get { return _studentassessment; }
set
{
    if (value != null && value.Length > 255)
        throw new ArgumentOutOfRangeException("StudentAssessment",
"StudentAssessment value, cannot contain more than 255 characters");
    _bIsChanged |= (_studentassessment != value);
    _studentassessment = value;
}
}
public virtual string Termid
{
    get { return _termid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Termid", "Termid value, cannot contain
more than 50 characters");
        _bIsChanged |= (_termid != value);
        _termid = value;
    }
}
public virtual string Classid
{
    get { return _classid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Classid", "Classid value, cannot
contain more than 50 characters");
        _bIsChanged |= (_classid != value);
        _classid = value;
    }
}
public virtual string Code
{
    get { return _code; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Code", "Code value, cannot contain more
than 50 characters");
        _bIsChanged |= (_code != value);
        _code = value;
    }
}
public virtual bool IsDeleted
{
    get
    {
```

```
return _bIsDeleted;
}
set
{
    _bIsDeleted = value;
}
}
public virtual bool IsChanged
{
    get
    {
        return _bIsChanged;
    }
    set
    {
        _bIsChanged = value;
    }
}
#endregion
#region Equals And GetHashCode Overrides
public override bool Equals( object obj )
{
    if( this == obj ) return true;
    ClassLesson castObj = null;
    try
    {
        castObj = (ClassLesson)obj;
    } catch(Exception) { return false; }
    return ( castObj != null ) &&
        ( this._clid == castObj.CliId );
}
public override int GetHashCode()
{
    int hash = 57;
    hash = 27 * hash * _clid.GetHashCode();
    return hash;
}
#endregion
#region ICloneable methods
public virtual object Clone()
{
    return this.MemberwiseClone();
}
#endregion
}
#region Custom ICollection interface for ClassLesson
public interface ICollectionLesson : ICollection
{
    ClassLesson this[int index]{get; set; }
```

```

void Add(ClassLesson pClassLesson);
void Clear();
}
[Serializable]
public class ClassLessonCollection : IClassLessonCollection
{
private IList<ClassLesson> _arrayInternal;
public ClassLessonCollection()
{
_arrayInternal = new List<ClassLesson>();
}
public ClassLessonCollection( IList<ClassLesson> pSource )
{
_arrayInternal = pSource;
if(_arrayInternal == null)
{
_arrayInternal = new List<ClassLesson>();
}
}
public ClassLesson this[int index]
{
get
{
return _arrayInternal[index];
}
set
{
_arrayInternal[index] = value;
}
}
public int Count { get { return _arrayInternal.Count; } }
public bool IsSynchronized { get { return false; } }
public object SyncRoot { get { return _arrayInternal; } }
public void CopyTo(Array array, int index) { _arrayInternal.CopyTo((ClassLesson[])array, index); }
public IEnumerator GetEnumerator() { return _arrayInternal.GetEnumerator(); }
public void Add(ClassLesson pClassLesson) { _arrayInternal.Add(pClassLesson); }
public void Clear() { _arrayInternal.Clear(); }
public IList<ClassLesson> GetList() { return _arrayInternal; }
}
#endregion
}
[Serializable]
public class ClassLessonLdx : ICloneable, IClassLessonLdx
{
#region Member Variables
protected string _id;
protected string _schoolid;
protected string _termid;
protected string _teacherid;

```

```
protected string _teachername;
protected string _remark;
protected bool _bIsDeleted;
protected bool _bIsChanged;
#endregion
#region Constructors
public ClassLessonLdx() {}
public ClassLessonLdx(string pId, string pSchoolid, string pTermid, string
pTeacherid, string pTeachername, string pRemark)
{
    this._id = pId;
    this._schoolid = pSchoolid;
    this._termid = pTermid;
    this._teacherid = pTeacherid;
    this._teachername = pTeachername;
    this._remark = pRemark;
}
public ClassLessonLdx(string pId)
{
    this._id = pId;
}
#endregion
#region Public Properties
public virtual string Id
{
    get { return _id; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Id", "Id value, cannot contain more than
50 characters");
        _bIsChanged |= (_id != value);
        _id = value;
    }
}
public virtual string Schoolid
{
    get { return _schoolid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Schoolid", "Schoolid value, cannot
contain more than 50 characters");
        _bIsChanged |= (_schoolid != value);
        _schoolid = value;
    }
}
}
using System;
using System.Collections.Generic;
using System.Linq;
```

```

using System.Text;
using System.Windows.Forms;
using System.Net;
using System.Net.Sockets;
public class CreditAttendanceRecordCollection : ICreditAttendanceRecordCollection
{
    private IList<CreditAttendanceRecord> _arrayInternal;
    public CreditAttendanceRecordCollection()
    {
        _arrayInternal = new List<CreditAttendanceRecord>();
    }
    public CreditAttendanceRecordCollection( IList<CreditAttendanceRecord> pSource )
    {
        _arrayInternal = pSource;
        if(_arrayInternal == null)
        {
            _arrayInternal = new List<CreditAttendanceRecord>();
        }
    }
    public int Count { get { return _arrayInternal.Count; } }
    public bool IsSynchronized { get { return false; } }
    public object SyncRoot { get { return _arrayInternal; } }
    public void CopyTo(Array array, int index) { _arrayInternal.CopyTo((CreditAttendanceRecord[])array, index); }
    public IEnumerator GetEnumerator() { return _arrayInternal.GetEnumerator(); }
    public void Add(CreditAttendanceRecord pCreditAttendanceRecord)
    { _arrayInternal.Add(pCreditAttendanceRecord); }
    public void Clear() { _arrayInternal.Clear(); }
    public IList<CreditAttendanceRecord> GetList() { return _arrayInternal; }
}
#endregion

using System;
using System.Collections;
using System.Collections.Generic;
[Serializable]
public class CreditAttendanceType : ICloneable, ICreditAttendanceType
{
    #region Member Variables
    protected int _id;
    protected string _name;
    protected string _isenable;
    protected string _schoolid;
    protected bool _bIsDeleted;
    protected bool _bIsChanged;
    #endregion
    #region Constructors
    public CreditAttendanceType() {}
    #endregion
    #region Equals And GetHashCode Overrides

```



```

public override bool Equals( object obj )
{
    if( this == obj ) return true;
    CreditAttendanceType castObj = null;
    try
    {
        castObj = (CreditAttendanceType)obj;
    } catch(Exception) { return false; }
    return ( castObj != null ) &&
        ( this._id == castObj.Id );
}

public override int GetHashCode()
{
    int hash = 57;
    hash = 27 * hash * _id.GetHashCode();
    return hash;
}

public virtual object Clone()
{
    return this.MemberwiseClone();
}
#endregion
}

#region Custom ICollection interface for CreditAttendanceType
public interface ICreditAttendanceTypeCollection : ICollection
{
    CreditAttendanceType this[int index]{get; set; }
    void Add(CreditAttendanceType pCreditAttendanceType);
    void Clear();
}

[Serializable]
public class CreditAttendanceTypeCollection : ICreditAttendanceTypeCollection
{
    private IList<CreditAttendanceType> _arrayInternal;
    public CreditAttendanceTypeCollection()
    {
        _arrayInternal = new List<CreditAttendanceType>();
    }
    public CreditAttendanceTypeCollection( IList<CreditAttendanceType> pSource )
    {
        _arrayInternal = pSource;
        if(_arrayInternal == null)
        {
            _arrayInternal = new List<CreditAttendanceType>();
        }
    }

    public int Count { get { return _arrayInternal.Count; } }
    public bool IsSynchronized { get { return false; } }
    public object SyncRoot { get { return _arrayInternal; } }
    public void CopyTo(Array array, int

```

```

index) { _arrayInternal.CopyTo((CreditAttendanceType[])array, index); }
public IEnumerator GetEnumerator() { return _arrayInternal.GetEnumerator(); }
public void Add(CreditAttendanceType pCreditAttendanceType)
{ _arrayInternal.Add(pCreditAttendanceType); }
public void Clear() { _arrayInternal.Clear(); }
public IList<CreditAttendanceType> GetList() { return _arrayInternal; }
}
#endregion
}
using System;
using System.Collections;
using System.Collections.Generic;
namespace OrmLib
{
[Serializable]
public class CreditClassType : ICloneable, ICreditClassType
{
#region Member Variables
protected int _id;
protected string _name;
protected string _isenable;
protected string _schoolid;
protected bool _bIsDeleted;
protected bool _bIsChanged;
#endregion
#region Constructors
public CreditClassType() {}
public virtual string Name
{
get { return _name; }
set
{
if (value != null && value.Length > 50)
throw new ArgumentOutOfRangeException("Name", "Name value, cannot contain more
than 50 characters");
_bIsChanged |= (_name != value);
_name = value;
}
}
public virtual string IsEnable
{
get { return _isenable; }
set
{
if (value != null && value.Length > 1)
throw new ArgumentOutOfRangeException("IsEnable", "IsEnable value, cannot
contain more than 1 characters");
_bIsChanged |= (_isenable != value);
_isenable = value;
}
}
}
}

```

```
}
public virtual string Schoolid
{
    get { return _schoolid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Schoolid", "Schoolid value, cannot
contain more than 50 characters");
        _bIsChanged |= (_schoolid != value);
        _schoolid = value;
    }
}
public override bool Equals( object obj )
{
    if( this == obj ) return true;
    CreditClassType castObj = null;
    try
    {
        castObj = (CreditClassType)obj;
    } catch(Exception) { return false; }
    return ( castObj != null ) &&
        ( this._id == castObj.Id );
}
public override int GetHashCode()
{
    int hash = 57;
    hash = 27 * hash * _id.GetHashCode();
    return hash;
}
public virtual object Clone()
{
    return this.MemberwiseClone();
}
#endregion
}
#region Custom ICollection interface for CreditClassType
public interface ICreditClassTypeCollection : ICollection
{
    CreditClassType this[int index]{get; set; }
    void Add(CreditClassType pCreditClassType);
    void Clear();
}
[Serializable]
public class CreditClassTypeCollection : ICreditClassTypeCollection
{
    private IList<CreditClassType> _arrayInternal;
    public CreditClassTypeCollection()
    {
        _arrayInternal = new List<CreditClassType>();
    }
}
```

```

}
public CreditClassTypeCollection( IList<CreditClassType> pSource )
{
    _arrayInternal = pSource;
    if(_arrayInternal == null)
    {
        _arrayInternal = new List<CreditClassType>();
    }
}

public int Count { get { return _arrayInternal.Count; } }
public bool IsSynchronized { get { return false; } }
public object SyncRoot { get { return _arrayInternal; } }
public void CopyTo(Array array, int index) { _arrayInternal.CopyTo((CreditClassType[])array, index); }
public IEnumerator GetEnumerator() { return _arrayInternal.GetEnumerator(); }
public void Add(CreditClassType pCreditClassType) { _arrayInternal.Add(pCreditClassType); }
public void Clear() { _arrayInternal.Clear(); }
public IList<CreditClassType> GetList() { return _arrayInternal; }
}
#endregion
}
[Serializable]
public class CreditCommunityService : ICloneable, ICreditCommunityService
{
    #region Member Variables
    protected int _id;
    protected string _studentname;
    protected string _studentid;
    protected string _classid;
    protected string _classname;
    protected DateTime? _servicedate;
    protected string _serviceobject;
    protected string _servicesubject;
    protected string _backtel;
    protected string _backperson;
    protected string _servicecontent;
    protected string _know;
    protected string _selfdegree;
    protected string _communitydegree;
    protected string _communityperson;
    protected string _parentdegree;
    protected string _parentperson;
    protected string _guidedegree;
    protected string _guideperson;
    protected double _creditscore;
    protected string _ispass;
    protected string _termid;
    protected string _schoolid;
    protected string _cultivationid;

```

```
protected string _cultivationname;
protected string _guidepersonid;
protected string _selfdegreecontent;
protected string _communitydegreecontent;
protected string _parentdegreecontent;
protected string _guidedegreecontent;
protected DateTime? _serviceenddate;
protected DateTime _applydate;
protected string _onrole;
protected string _teammember;
protected string _teamnumber;
protected string _teammembernames;
protected double _total;
protected string _teamdegrescontent;
protected bool _bIsDeleted;
protected bool _bIsChanged;
#endregion
#region Constructors
public CreditCommunityService() {}
public CreditCommunityService(string pStudentName, string pStudentId, string
pClassId, string pClassName, DateTime? pServiceDate, string pServiceObject, string
pServiceSubject, string pBackTel, string pBackPerson, string pServiceContent,
string pKnow, string pSelfDegree, string pCommunityDegree, string pCommunityPerson,
string pParentDegree, string pParentPerson, string pGuideDegree, string
pGuidePerson, double pCreditScore, string pIsPass, string pTermId, string pSchoolId,
string pCultivationId, string pCultivationName, string pGuidePersonId, string
pSelfDegreeContent, string pCommunityDegreeContent, string pParentDegreeContent,
string pGuideDegreeContent, DateTime? pServiceEndDate, DateTime pApplyDate, string
pOnRole, string pTeamMember, string pTeamNumber, string pTeamMemberNames, double
pTotal, string pTeamDegresContent)
{
this._studentname = pStudentName;
this._studentid = pStudentId;
this._classid = pClassId;
this._classname = pClassName;
this._servicedate = pServiceDate;
this._serviceobject = pServiceObject;
this._servicesubject = pServiceSubject;
this._backtel = pBackTel;
this._backperson = pBackPerson;
this._servicecontent = pServiceContent;
this._know = pKnow;
this._selfdegree = pSelfDegree;
this._communitydegree = pCommunityDegree;
this._communityperson = pCommunityPerson;
this._parentdegree = pParentDegree;
this._parentperson = pParentPerson;
this._guidedegree = pGuideDegree;
this._guideperson = pGuidePerson;
this._creditscore = pCreditScore;
```

```
this._ispass = pIsPass;
this._termid = pTermId;
this._schoolid = pSchoolId;
this._cultivationid = pCultivationId;
this._cultivationname = pCultivationName;
this._guidepersonid = pGuidePersonId;
this._selfdegreecontent = pSelfDegreeContent;
this._communitydegreecontent = pCommunityDegreeContent;
this._parentdegreecontent = pParentDegreeContent;
this._guidedegreecontent = pGuideDegreeContent;
this._serviceenddate = pServiceEndDate;
this._applydate = pApplyDate;
this._onrole = pOnRole;
this._teammember = pTeamMember;
this._teamnumber = pTeamNumber;
this._teammembersnames = pTeamMemberNames;
this._total = pTotal;
this._teamdegrescontent = pTeamDegresContent;
}

public virtual string StudentName
{
    get { return _studentname; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("StudentName", "StudentName value,
cannot contain more than 50 characters");
        _bIsChanged |= (_studentname != value);
        _studentname = value;
    }
}

public virtual string StudentId
{
    get { return _studentid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("StudentId", "StudentId value, cannot
contain more than 50 characters");
        _bIsChanged |= (_studentid != value);
        _studentid = value;
    }
}

public virtual string ClassId
{
    get { return _classid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("ClassId", "ClassId value, cannot
```

```
contain more than 50 characters");
    _bIsChanged |= (_classid != value);
    _classid = value;
}
}
public virtual string ClassName
{
    get { return _classname; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("ClassName", "ClassName value, cannot
contain more than 50 characters");
        _bIsChanged |= (_classname != value);
        _classname = value;
    }
}
public virtual DateTime? ServiceDate
{
    get { return _servicedate; }
    set { _bIsChanged |= (_servicedate != value); _servicedate = value; }
}
public virtual string ServiceObject
{
    get { return _serviceobject; }
    set
    {
        if (value != null && value.Length > 1073741823)
            throw new ArgumentOutOfRangeException("ServiceObject", "ServiceObject value,
cannot contain more than 1073741823 characters");
        _bIsChanged |= (_serviceobject != value);
        _serviceobject = value;
    }
}
public virtual string ServiceSubject
{
    get { return _servicesubject; }
    set
    {
        if (value != null && value.Length > 1073741823)
            throw new ArgumentOutOfRangeException("ServiceSubject", "ServiceSubject value,
cannot contain more than 1073741823 characters");
        _bIsChanged |= (_servicesubject != value);
        _servicesubject = value;
    }
}
public virtual string BackTel
{
    get { return _backtel; }
    set
```

```
{
    if (value != null && value.Length > 50)
        throw new ArgumentOutOfRangeException("BackTel", "BackTel value, cannot
contain more than 50 characters");
    _bIsChanged |= (_backtel != value);
    _backtel = value;
}
}
public virtual string BackPerson
{
    get { return _backperson; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("BackPerson", "BackPerson value, cannot
contain more than 50 characters");
        _bIsChanged |= (_backperson != value);
        _backperson = value;
    }
}
public virtual string ServiceContent
{
    get { return _servicecontent; }
    set
    {
        if (value != null && value.Length > 1073741823)
            throw new ArgumentOutOfRangeException("ServiceContent", "ServiceContent value,
cannot contain more than 1073741823 characters");
        _bIsChanged |= (_servicecontent != value);
        _servicecontent = value;
    }
}
public virtual string Know
{
    set
    {
        if (value != null && value.Length > 1073741823)
            throw new ArgumentOutOfRangeException("Know", "Know value, cannot contain more
than 1073741823 characters");
        _bIsChanged |= (_know != value);
        _know = value;
    }
}
public virtual string SelfDegree
{
    get { return _selfdegree; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("SelfDegree", "SelfDegree value, cannot
```



```
contain more than 50 characters");
    _bIsChanged |= (_selfdegree != value);
    _selfdegree = value;
}
}
public virtual string CommunityDegree
{
    get { return _communitydegree; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("CommunityDegree", "CommunityDegree
value, cannot contain more than 50 characters");
        _bIsChanged |= (_communitydegree != value);
        _communitydegree = value;
    }
}
public virtual string CommunityPerson
{
    get { return _communityperson; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("CommunityPerson", "CommunityPerson
value, cannot contain more than 50 characters");
        _bIsChanged |= (_communityperson != value);
        _communityperson = value;
    }
}
public virtual string ParentDegree
{
    get { return _parentdegree; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("ParentDegree", "ParentDegree value,
cannot contain more than 50 characters");
        _bIsChanged |= (_parentdegree != value);
        _parentdegree = value;
    }
}
public virtual string ParentPerson
{
    get { return _parentperson; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("ParentPerson", "ParentPerson value,
cannot contain more than 50 characters");
        _bIsChanged |= (_parentperson != value);
```

```
    _parentperson = value;
}
}
public virtual string GuideDegree
{
    get { return _guidedegree; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("GuideDegree", "GuideDegree value,
cannot contain more than 50 characters");
        _bIsChanged |= (_guidedegree != value);
        _guidedegree = value;
    }
}
public virtual string GuidePerson
{
    get { return _guideperson; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("GuidePerson", "GuidePerson value,
cannot contain more than 50 characters");
        _bIsChanged |= (_guideperson != value);
        _guideperson = value;
    }
}
public virtual double CreditScore
{
    get { return _creditscore; }
    set { _bIsChanged |= (_creditscore != value); _creditscore = value; }
}
public virtual string IsPass
{
    get { return _ispass; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("IsPass", "IsPass value, cannot contain
more than 50 characters");
        _bIsChanged |= (_ispass != value);
        _ispass = value;
    }
}
public virtual string TermId
{
    get { return _termid; }
    set
    {
        if (value != null && value.Length > 50)
```

```
        throw new ArgumentOutOfRangeException("TermId", "TermId value, cannot contain
more than 50 characters");
        _bIsChanged |= (_termid != value);
        _termid = value;
    }
}

public virtual string SchoolId
{
    get { return _schoolid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("SchoolId", "SchoolId value, cannot
contain more than 50 characters");
        _bIsChanged |= (_schoolid != value);
        _schoolid = value;
    }
}

public virtual string CultivationId
{
    get { return _cultivationid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("CultivationId", "CultivationId value,
cannot contain more than 50 characters");
        _bIsChanged |= (_cultivationid != value);
        _cultivationid = value;
    }
}

public virtual string CultivationName
{
    get { return _cultivationname; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("CultivationName", "CultivationName
value, cannot contain more than 50 characters");
        _bIsChanged |= (_cultivationname != value);
        _cultivationname = value;
    }
}

public virtual string GuidePersonId
{
    get { return _guidepersonid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("GuidePersonId", "GuidePersonId value,
cannot contain more than 50 characters");
```

```
        _bIsChanged |= (_guidepersonid != value);
        _guidepersonid = value;
    }
}

public virtual string SelfDegreeContent
{
    get { return _selfdegreecontent; }
    set
    {
        if (value != null && value.Length > 1073741823)
            throw new ArgumentOutOfRangeException("SelfDegreeContent",
"SelfDegreeContent value, cannot contain more than 1073741823 characters");
        _bIsChanged |= (_selfdegreecontent != value);
        _selfdegreecontent = value;
    }
}

public virtual string CommunityDegreeContent
{
    get { return _communitydegreecontent; }
    set
    {
        if (value != null && value.Length > 1073741823)
            throw new ArgumentOutOfRangeException("CommunityDegreeContent",
"CommunityDegreeContent value, cannot contain more than 1073741823 characters");
        _bIsChanged |= (_communitydegreecontent != value);
        _communitydegreecontent = value;
    }
}

public virtual string ParentDegreeContent
{
    get { return _parentdegreecontent; }
    set
    {
        if (value != null && value.Length > 1073741823)
            throw new ArgumentOutOfRangeException("ParentDegreeContent",
"ParentDegreeContent value, cannot contain more than 1073741823 characters");
        _bIsChanged |= (_parentdegreecontent != value);
        _parentdegreecontent = value;
    }
}

public virtual string GuideDegreeContent
{
    get { return _guidedegreecontent; }
    set
    {
        if (value != null && value.Length > 1073741823)
            throw new ArgumentOutOfRangeException("GuideDegreeContent",
"GuideDegreeContent value, cannot contain more than 1073741823 characters");
        _bIsChanged |= (_guidedegreecontent != value);
        _guidedegreecontent = value;
    }
}
```

```
}
}
public virtual DateTime? ServiceEndDate
{
    get { return _serviceenddate; }
    set { _bIsChanged |= (_serviceenddate != value); _serviceenddate = value; }
}
public virtual DateTime ApplyDate
{
    get { return _applydate; }
    set { _bIsChanged |= (_applydate != value); _applydate = value; }
}
public virtual string OnRole
{
    get { return _onrole; }
    set
    {
        if (value != null && value.Length > 1)
            throw new ArgumentOutOfRangeException("OnRole", "OnRole value, cannot contain more than 1 characters");
        _bIsChanged |= (_onrole != value);
        _onrole = value;
    }
}
public virtual string TeamMember
{
    get { return _teammember; }
    set
    {
        if (value != null && value.Length > 1073741823)
            throw new ArgumentOutOfRangeException("TeamMember", "TeamMember value, cannot contain more than 1073741823 characters");
        _bIsChanged |= (_teammember != value);
        _teammember = value;
    }
}
public virtual string TeamNumber
{
    get { return _teamnumber; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("TeamNumber", "TeamNumber value, cannot contain more than 50 characters");
        _bIsChanged |= (_teamnumber != value);
        _teamnumber = value;
    }
}
public virtual string TeamMemberNames
{

```

```
get { return _teammembernames; }
set
{
    if (value != null && value.Length > 1073741823)
        throw new ArgumentOutOfRangeException("TeamMemberNames", "TeamMemberNames
value, cannot contain more than 1073741823 characters");
    _bIsChanged |= (_teammembernames != value);
    _teammembernames = value;
}
}
public virtual double Total
{
    get { return _total; }
    set { _bIsChanged |= (_total != value); _total = value; }
}
public virtual string TeamDegresContent
{
    get { return _teamdegrescontent; }
    set
    {
        if (value != null && value.Length > 1073741823)
            throw new ArgumentOutOfRangeException("TeamDegresContent",
"TeamDegresContent value, cannot contain more than 1073741823 characters");
        _bIsChanged |= (_teamdegrescontent != value);
        _teamdegrescontent = value;
    }
}
public override bool Equals( object obj )
{
    if( this == obj ) return true;
    CreditCommunityService castObj = null;
    try
    {
        castObj = (CreditCommunityService)obj;
    } catch(Exception) { return false; }
    return ( castObj != null ) &&
        ( this._id == castObj.Id );
}
public override int GetHashCode()
{
    int hash = 57;
    hash = 27 * hash * _id.GetHashCode();
    return hash;
}
public virtual object Clone()
{
    return this.MemberwiseClone();
}
#region Custom ICollection interface for CreditCommunityService
public interface ICreditCommunityServiceCollection : ICollection
```

```

{
    CreditCommunityService this[int index]{get; set; }
    void Add(CreditCommunityService pCreditCommunityService);
    void Clear();
}
[Serializable]
public class CreditCommunityServiceCollection : ICreditCommunityServiceCollection
{
    private IList<CreditCommunityService> _arrayInternal;
    public CreditCommunityServiceCollection()
    {
        _arrayInternal = new List<CreditCommunityService>();
    }
    public CreditCommunityServiceCollection( IList<CreditCommunityService> pSource )
    {
        _arrayInternal = pSource;
        if(_arrayInternal == null)
        {
            _arrayInternal = new List<CreditCommunityService>();
        }
    }
    public CreditCommunityService this[int index]
    {
        get
        {
            return _arrayInternal[index];
        }
        set
        {
            _arrayInternal[index] = value;
        }
    }
    public int Count { get { return _arrayInternal.Count; } }
    public bool IsSynchronized { get { return false; } }
    public object SyncRoot { get { return _arrayInternal; } }
    public void CopyTo(Array array, int index){ _arrayInternal.CopyTo((CreditCommunityService[])array, index); }
    public IEnumerator GetEnumerator() { return _arrayInternal.GetEnumerator(); }
    public void Add(CreditCommunityService pCreditCommunityService)
    { _arrayInternal.Add(pCreditCommunityService); }
    public void Clear() { _arrayInternal.Clear(); }
    public IList<CreditCommunityService> GetList() { return _arrayInternal; }
}
#endregion
}
using System;
using System.Collections;
using System.Collections.Generic;
namespace OrmLib
{

```

```
[Serializable]
public class CreditComQualityEvalition : ICloneable, ICreditComQualityEvalition
{
    #region Member Variables
    protected int _id;
    protected string _name;
    protected string _schoolid;
    protected string _termid;
    protected int? _stairid;
    protected string _stairname;
    protected int? _secendid;
    protected string _secendname;
    protected string _section;
    protected bool _bIsDeleted;
    protected bool _bIsChanged;
    #endregion
    #region Constructors
    public CreditComQualityEvalition() {}
    public CreditComQualityEvalition(string pName, string pSchoolId, string pTermId,
    int? pStairId, string pStairName, int? pSecendId, string pSecendName, string
    pSection)
    {
        this._name = pName;
        this._schoolid = pSchoolId;
        this._termid = pTermId;
        this._stairid = pStairId;
        this._stairname = pStairName;
        this._secendid = pSecendId;
        this._secendname = pSecendName;
        this._section = pSection;
    }
    public virtual string Name
    {
        get { return _name; }
        set
        {
            if (value != null && value.Length > 500)
                throw new ArgumentOutOfRangeException("Name", "Name value, cannot contain more
                than 500 characters");
            _bIsChanged |= (_name != value);
            _name = value;
        }
    }
    public virtual string SchoolId
    {
        get { return _schoolid; }
        set
        {
            if (value != null && value.Length > 50)
                throw new ArgumentOutOfRangeException("SchoolId", "SchoolId value, cannot
```



```
contain more than 50 characters");
    _bIsChanged |= (_schoolid != value);
    _schoolid = value;
}
}
public virtual string TermId
{
    get { return _termid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("TermId", "TermId value, cannot contain
more than 50 characters");
        _bIsChanged |= (_termid != value);
        _termid = value;
    }
}
public virtual int? StairId
{
    get { return _stairid; }
    set { _bIsChanged |= (_stairid != value); _stairid = value; }
}
public virtual string StairName
{
    get { return _stairname; }
    set
    {
        if (value != null && value.Length > 500)
            throw new ArgumentOutOfRangeException("StairName", "StairName value, cannot
contain more than 500 characters");
        _bIsChanged |= (_stairname != value);
        _stairname = value;
    }
}
public virtual int? SecendId
{
    get { return _secendid; }
    set { _bIsChanged |= (_secendid != value); _secendid = value; }
}
public virtual string SecendName
{
    get { return _secendname; }
    set
    {
        if (value != null && value.Length > 500)
            throw new ArgumentOutOfRangeException("SecendName", "SecendName value, cannot
contain more than 500 characters");
        _bIsChanged |= (_secendname != value);
        _secendname = value;
    }
}
```

```

}
public virtual string Section
{
    get { return _section; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Section", "Section value, cannot
contain more than 50 characters");
        _bIsChanged |= (_section != value);
        _section = value;
    }
}
#endregion
#region Equals And GetHashCode Overrides
public override bool Equals( object obj )
{
    if( this == obj ) return true;
    CreditComQualityEvalition castObj = null;
    try
    {
        castObj = (CreditComQualityEvalition)obj;
    } catch(Exception) { return false; }
    return ( castObj != null ) &&
        ( this._id == castObj.Id );
}
public override int GetHashCode()
{
    int hash = 57;
    hash = 27 * hash * _id.GetHashCode();
    return hash;
}
#endregion
#region ICloneable methods
public virtual object Clone()
{
    return this.MemberwiseClone();
}
#endregion
}
#region Custom ICollection interface for CreditComQualityEvalition
public interface ICreditComQualityEvalitionCollection : ICollection
{
    CreditComQualityEvalition this[int index]{get; set; }
    void Add(CreditComQualityEvalition pCreditComQualityEvalition);
    void Clear();
}
[Serializable]
public class CreditComQualityEvalitionCollection :
    ICreditComQualityEvalitionCollection

```

```

{
private IList<CreditComQualityEvalition> _arrayInternal;
public CreditComQualityEvalitionCollection()
{
_arrayInternal = new List<CreditComQualityEvalition>();
}
public CreditComQualityEvalitionCollection( IList<CreditComQualityEvalition>
pSource )
{
_arrayInternal = pSource;
if(_arrayInternal == null)
{
_arrayInternal = new List<CreditComQualityEvalition>();
}
}
public CreditComQualityEvalition this[int index]
{
get
{
return _arrayInternal[index];
}
set
{
_arrayInternal[index] = value;
}
}
public int Count { get { return _arrayInternal.Count; } }
public bool IsSynchronized { get { return false; } }
public object SyncRoot { get { return _arrayInternal; } }
public void CopyTo(Array array, int
index){ _arrayInternal.CopyTo((CreditComQualityEvalition[])array, index); }
public IEnumerator GetEnumerator() { return _arrayInternal.GetEnumerator(); }
public void Add(CreditComQualityEvalition pCreditComQualityEvalition)
{ _arrayInternal.Add(pCreditComQualityEvalition); }
public void Clear() { _arrayInternal.Clear(); }
public IList<CreditComQualityEvalition> GetList() { return _arrayInternal; }
}
#endregion
}

public ContractInfo(string pId, string pContractNo, string pContractName, decimal?
pMoney, string pCurrency, DateTime? pDodate, string pManname, string pManid, string
pDeptid, string pDeptname, string pZfkind, DateTime? pSdate, DateTime? pEdate,
string pZftype, string pContractContent, string pContractDetail, string
pContractBug, string pCompanya, string pCompanyAid, string pCompanyAman, string
pCompanyb, string pCompanyBid, string pCompanyBman, string pIsSave, string pAddress,
string pState, string pRemark, DateTime? pIndate, string pInman, string pInmanid,
string pSchoolid, string pBuykind, string pBuydept)
{
this._id = pId;
this._contractno = pContractNo;

```

```
this._contractname = pContractName;
this._money = pMoney;
this._currency = pCurrency;
this._dodate = pDodate;
this._manname = pManname;
this._manid = pManid;
this._deptid = pDeptid;
this._deptname = pDeptname;
this._zfkind = pZfkind;
this._sdate = pSdate;
this._edate = pEdate;
this._zftype = pZftype;
this._contractcontent = pContractContent;
this._contractdetail = pContractDetail;
this._contractbug = pContractBug;
this._companya = pCompanya;
this._companyaid = pCompanyAid;
this._companyaman = pCompanyAman;
this._companyb = pCompanyb;
this._companybid = pCompanyBid;
this._companybman = pCompanyBman;
this._issave = pIsSave;
this._address = pAddress;
this._state = pState;
this._remark = pRemark;
this._indate = pIndate;
this._inman = pInman;
this._inmanid = pInmanid;
this._schoolid = pSchoolid;
this._buykind = pBuykind;
this._buydept = pBuydept;
}
public ContractInfo(string pId)
{
    this._id = pId;
}
#endregion
#region Public Properties
public virtual string Id
{
    get { return _id; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Id", "Id value, cannot contain more than
50 characters");
        _IsChanged |= (_id != value);
        _id = value;
    }
}
}
```

```
public virtual string ContractNo
{
    get { return _contractno; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("ContractNo", "ContractNo value, cannot
contain more than 50 characters");
        _bIsChanged |= (_contractno != value);
        _contractno = value;
    }
}

public virtual string ContractName
{
    get { return _contractname; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("ContractName", "ContractName value,
cannot contain more than 50 characters");
        _bIsChanged |= (_contractname != value);
        _contractname = value;
    }
}

public virtual decimal? Money
{
    get { return _money; }
    set { _bIsChanged |= (_money != value); _money = value; }
}

public virtual string Currency
{
    get { return _currency; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Currency", "Currency value, cannot
contain more than 50 characters");
        _bIsChanged |= (_currency != value);
        _currency = value;
    }
}

public virtual DateTime? Dodate
{
    get { return _dodate; }
    set { _bIsChanged |= (_dodate != value); _dodate = value; }
}

public virtual string Manname
{
    get { return _manname; }
    set
```

```
{
    if (value != null && value.Length > 50)
        throw new ArgumentOutOfRangeException("Manname", "Manname value, cannot contain more than 50 characters");
    _bIsChanged |= (_manname != value);
    _manname = value;
}
}

public virtual string Manid
{
    get { return _manid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Manid", "Manid value, cannot contain more than 50 characters");
        _bIsChanged |= (_manid != value);
        _manid = value;
    }
}

public virtual string Deptid
{
    get { return _deptid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Deptid", "Deptid value, cannot contain more than 50 characters");
        _bIsChanged |= (_deptid != value);
        _deptid = value;
    }
}

public virtual string Deptname
{
    get { return _deptname; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Deptname", "Deptname value, cannot contain more than 50 characters");
        _bIsChanged |= (_deptname != value);
        _deptname = value;
    }
}

public virtual string Zfkind
{
    get { return _zfkind; }
    set
    {
        if (value != null && value.Length > 50)
```

```
        throw new ArgumentOutOfRangeException("Zfkind", "Zfkind value, cannot contain
more than 50 characters");
        _bIsChanged |= (_zfkind != value);
        _zfkind = value;
    }
}

public virtual DateTime? Sdate
{
    get { return _sdate; }
    set { _bIsChanged |= (_sdate != value); _sdate = value; }
}

public virtual DateTime? Edate
{
    get { return _edate; }
    set { _bIsChanged |= (_edate != value); _edate = value; }
}

public virtual string Zftype
{
    get { return _zftype; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Zftype", "Zftype value, cannot contain
more than 50 characters");
        _bIsChanged |= (_zftype != value);
        _zftype = value;
    }
}

public virtual string ContractContent
{
    get { return _contractcontent; }
    set
    {
        if (value != null && value.Length > 4000)
            throw new ArgumentOutOfRangeException("ContractContent", "ContractContent
value, cannot contain more than 4000 characters");
        _bIsChanged |= (_contractcontent != value);
        _contractcontent = value;
    }
}

public virtual string ContractDetail
{
    get { return _contractdetail; }
    set
    {
        if (value != null && value.Length > 2147483647)
            throw new ArgumentOutOfRangeException("ContractDetail", "ContractDetail value,
cannot contain more than 2147483647 characters");
        _bIsChanged |= (_contractdetail != value);
        _contractdetail = value;
    }
}
```

```
}
}
public virtual string ContractBug
{
    get { return _contractbug; }
    set
    {
        if (value != null && value.Length > 4000)
            throw new ArgumentOutOfRangeException("ContractBug", "ContractBug value, cannot contain more than 4000 characters");
        _bIsChanged |= (_contractbug != value);
        _contractbug = value;
    }
}
public virtual string Companya
{
    get { return _companya; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Companya", "Companya value, cannot contain more than 50 characters");
        _bIsChanged |= (_companya != value);
        _companya = value;
    }
}
public virtual string CompanyAid
{
    get { return _companyaid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("CompanyAid", "CompanyAid value, cannot contain more than 50 characters");
        _bIsChanged |= (_companyaid != value);
        _companyaid = value;
    }
}
public virtual string CompanyAman
{
    get { return _companyaman; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("CompanyAman", "CompanyAman value, cannot contain more than 50 characters");
        _bIsChanged |= (_companyaman != value);
        _companyaman = value;
    }
}
```



```
public virtual string Companyb
{
    get { return _companyb; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Companyb", "Companyb value, cannot
contain more than 50 characters");
        _bIsChanged |= (_companyb != value);
        _companyb = value;
    }
}

public virtual string CompanyBid
{
    get { return _companybid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("CompanyBid", "CompanyBid value, cannot
contain more than 50 characters");
        _bIsChanged |= (_companybid != value);
        _companybid = value;
    }
}

public virtual string CompanyBman
{
    get { return _companybman; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("CompanyBman", "CompanyBman value,
cannot contain more than 50 characters");
        _bIsChanged |= (_companybman != value);
        _companybman = value;
    }
}

public virtual string Buykind
{
    get { return _buykind; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Buykind", "Buykind value, cannot
contain more than 50 characters");
        _bIsChanged |= (_buykind != value);
        _buykind = value;
    }
}

public virtual string Buydept
{

```

```
get { return _buydept; }
set
{
    if (value != null && value.Length > 50)
        throw new ArgumentOutOfRangeException("Buydept", "Buydept value, cannot
contain more than 50 characters");
    _bIsChanged |= (_buydept != value);
    _buydept = value;
}
public virtual string IsSave
{
    get { return _issave; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("IsSave", "IsSave value, cannot contain
more than 50 characters");
        _bIsChanged |= (_issave != value);
        _issave = value;
    }
}
public virtual string Address
{
    get { return _address; }
    set
    {
        if (value != null && value.Length > 100)
            throw new ArgumentOutOfRangeException("Address", "Address value, cannot
contain more than 100 characters");
        _bIsChanged |= (_address != value);
        _address = value;
    }
}
public virtual string State
{
    get { return _state; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("State", "State value, cannot contain
more than 50 characters");
        _bIsChanged |= (_state != value);
        _state = value;
    }
}
public virtual string Remark
{
    get { return _remark; }
    set
    {
```

```
        if (value != null && value.Length > 500)
            throw new ArgumentOutOfRangeException("Remark", "Remark value, cannot contain
more than 500 characters");
        _bIsChanged |= (_remark != value);
        _remark = value;
    }
}

public virtual DateTime? Indate
{
    get { return _indate; }
    set { _bIsChanged |= (_indate != value); _indate = value; }
}

public virtual string Inman
{
    get { return _inman; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Inman", "Inman value, cannot contain
more than 50 characters");
        _bIsChanged |= (_inman != value);
        _inman = value;
    }
}

public virtual string Inmanid
{
    get { return _inmanid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Inmanid", "Inmanid value, cannot
contain more than 50 characters");
        _bIsChanged |= (_inmanid != value);
        _inmanid = value;
    }
}

public virtual string Schoolid
{
    get { return _schoolid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Schoolid", "Schoolid value, cannot
contain more than 50 characters");
        _bIsChanged |= (_schoolid != value);
        _schoolid = value;
    }
}

public virtual string Buykind
{

```

```
get { return _buykind; }
set
{
    if (value != null && value.Length > 50)
        throw new ArgumentOutOfRangeException("Buykind", "Buykind value, cannot
contain more than 50 characters");
    _bIsChanged |= (_buykind != value);
    _buykind = value;
}
}
public virtual string Buydept
{
get { return _buydept; }
set
{
    if (value != null && value.Length > 50)
        throw new ArgumentOutOfRangeException("Buydept", "Buydept value, cannot
contain more than 50 characters");
    _bIsChanged |= (_buydept != value);
    _buydept = value;
}
}
public class CultivateArticleWork : ICloneable, ICultivateArticleWork
{
#region Member Variables
protected string _id;
protected string _title;
#endregion
#region Constructors
public CultivateArticleWork() {}
public CultivateArticleWork(string pId, string pTitle, string pContent, string
pUserCode, string pUserName, DateTime? pCreateTime, string pArticleId, double
pScore, string pFilePath, string pSchoolId)
{
this._id = pId;
this._title = pTitle;
this._content = pContent;
this._usercode = pUserCode;
this._username = pUserName;
this._filepath = pFilePath;
this._schoolid = pSchoolId;
}
public CultivateArticleWork(string pId)
{
this._id = pId;
}
}
```